

Lehrdemonstrationen zur digitalen Filterung auf eingebetteten Systemen

Eduard Funkner

Martin Schwarz

2025-06-24

Inhaltsverzeichnis

Theoretische Grundlagen	3
Infinite Impulse Response (IIR)-Filter	3
Differenzgleichung eines IIR Filters zweiter Ordnung	3
Übertragungsfunktion eines IIR-Filters zweiter Ordnung	4
Implementierungsstrukturen biquadratischer Filter	5
Direktform 1	5
Direktform 2 (Kanonische Form)	6
Transponierte Direktform 2:	7
Implementierungserklärung der Biquad Filter in Arduino	9
Filter Basisklasse	9
Direktform 1	9
Direktform 2	10
Koeffizienten	12
Filter Anwendung	12

Theoretische Grundlagen

Infinite Impulse Response (IIR)-Filter

Infinite Impulse Response (IIR) Filter gehören in der Signalverarbeitung zu den grundlegenden Typen der digitalen Filter. IIR-Filter zeichnen sich fundamental durch ihre charakteristische Ausprägung aus, bei der Berechnung des aktuellen Ausgangswertes durch die Verwendung des gegenwärtigen und vergangenen Eingabewertes als auch des vorherigen Ausgabewertes. Aufgrund dieser Rückkopplung kann es dazu führen, dass die Impulsantwort von IIR-Filtern theoretisch unendlich lang andauern kann, was sie grundsätzlich von Finite Impulse Response (FIR) Filtern unterscheidet.

Ein immenser Vorteil von IIR-Filtern liegt in der hohen Effizienz bei der Realisierung scharfter Frequenzgänge mit relativ wenigen Koeffizienten, während FIR-Filter für vergleichbare Filtereigenschaften oft hunderte von Koeffizienten benötigen, können IIR-Filter ähnliche Ergebnisse mit deutlich geringerem Rechenaufwand erreichen. Aufgrund dieser Eigenschaft eignen sich IIR-Filter besonders gut im Einsatz von Systemen mit beschränkten Ressourcen oder in Echtzeitverarbeitung.

Biquadratische Filter, kurz als "Biquad" bezeichnet, sind eine spezielle Unterklasse von IIR-Filtern zweiter Ordnung. Durch das Kaskadieren solcher Biquad-Sektionen besteht die Möglichkeit komplexe IIR-Filter höherer Ordnung auf einfache Weise zu realisieren. Das Kaskadieren dieser Sektionen bringt Vorteile in Bezug auf numerische Stabilität, Flexibilität bei der Parameteranpassung und Modularität des Designs.

Differenzengleichung eines IIR Filters zweiter Ordnung

Digitale Filter können mit einer linearen Differenzengleichung beschrieben werden. Die biquadratischen Filter werden mit der Differenzengleichung eines IIR-Filters zweiter Ordnung beschrieben werden:

$$y[n] = \frac{1}{a_0} (b_0 \cdot x[n] + b_1 \cdot x[n-1] + b_2 \cdot x[n-2] - a_1 \cdot y[n-1] - a_2 \cdot y[n-2])$$

Der Wert $y[n]$ bezeichnet den Ausgabewert zum Sample n , während $x[n]$ den Eingangswert darstellt. Der aktuelle Ausgabewert $y[n]$ ergibt sich aus der gewichteten Summe der aktuellen und zwei vergangenen Eingabewerte, subtrahiert mit den zwei gewichteten vergangenen Ausgabewerte. Die Koeffizienten b_0 , b_1 und b_2 bestimmen den Einfluss des aktuellen und der vergangenen

Eingabewerte (Feedforward), während a_1 und a_2 den Rückkopplungsanteil aus früheren Ausgabewerten (Feedback) modellieren. Im Allgemeinen wird der Koeffizient auf a_0 auf 1 normiert:

$$y[n] = b_0 \cdot x[n] + b_1 \cdot x[n-1] + b_2 \cdot x[n-2] - a_1 \cdot y[n-1] - a_2 \cdot y[n-2]$$

Zur Analyse des Frequenz- und Phasenverhaltens des Filters, wird die Differenzengleichung mittels der z-Transformation in den z-Bereich transformiert und daraus folgt die folgende Übertragungsfunktion.

Übertragungsfunktion eines IIR-Filters zweiter Ordnung

$$H(z) = \frac{Y(z)}{X(z)} = \frac{b_0 + b_1 z^{-1} + b_2 z^{-2}}{1 + a_1 z^{-1} + a_2 z^{-2}}$$

Die Übertragungsfunktion eines Biquads beschreibt das Verhältnis zwischen dem Ausgang $Y(z)$ zu dem Eingang $X(z)$. Diese Gleichung stellt die normierte Form des Biquads dar, bei welcher der Nennerkoeffizient auf $a_0 = 1$ normiert ist. Für den Fall, dass $a_0 \neq 1$ beträgt müssen die Koeffizienten durch a_0 geteilt werden um die Normalform zu erreichen.

Anhand der Übertragungsfunktion können die Pol- und Nullstellen des Filters analysiert werden, um die Stabilität des Biquad-Filters zu vergewissern. Dabei müssen alle Polstellen des Filters innerhalb des Einheitskreises der z-Ebene liegen, beziehungsweise der Betrag jedes Pols kleiner als 1 sein muss.

Befinden sich Pole außerhalb oder direkt auf dem Einheitskreis, kann es dazu kommen dass das Ausgangssignal unkontrolliert schwingt und das System damit instabil wird. Nullstellen haben hingegen lediglich Einfluss auf die Frequenzcharakteristik und nicht direkt auf die Stabilität.

Implementierungsstrukturen biquadratischer Filter

Die praktische Implementierung von biquadratischen Filtern erfolgt durch die direkte Umsetzung der Differenzengleichungen in Software oder Hardware.

Dafür werden die verzögerten Eingangswerte $x[n-1]$ und $x[n-2]$, sowie die Ausgangswerte $y[n-1]$ und $y[n-2]$ in Speicherelementen, sogenannten Verzögerungsgliedern, gespeichert. Der aktuelle Ausgangswert $y[n]$ wird mittels der gewichteten Summation aller Terme gemäß der jeweiligen Differenzengleichung berechnet.

Bei der numerischen Implementierung sind verschiedene Aspekte zu berücksichtigen, insbesondere die Wortbreite und des Zahlenformats (Festkomma oder Gleitkomma).

Weitere Einflüsse wie Rundungsfehler oder Quantisierungsrauschen können die Filtercharakteristik beeinflussen und im Extremfall die Stabilität des Filters gefährden.

Diese Implementierungsstrukturen bieten die Möglichkeit mehrere Filter-Sektionen aneinander zu Verbinden. Diese Modularität biquadratischer Filter ermöglicht, komplexere Filtersysteme höherer Ordnung, durch die Kaskadierung mehrerer Biquad-Sektionen zu realisieren.

Dadurch können Vorteile im Bezug auf numerische Stabilität, Designflexibilität und Implementierungseffizienz geschaffen werden, weil jede Sektion unabhängig entworfen und optimiert werden kann.

Direktform 1

Die Direktform 1 stellt die direkte Umsetzung der biquadratischen Differenzengleichung in einer Signalfussstruktur dar.

Die Implementierung folgt direkt der mathematischen Beschreibung unter der Verwendung separater Verzögerungsketten für die Eingangs- als auch die Ausgangswerte.

Diese Struktur kann als eine Parallelschaltung eines rein rekursiven Teils (nur Pole) und eines nicht rekursiven Teils (nur Nullstellen) aufgefasst werden.

$$y[n] = b_0 \cdot x[n] + b_1 \cdot x[n-1] + b_2 \cdot x[n-2] - a_1 \cdot y[n-1] - a_2 \cdot y[n-2]$$

Für die Direktform 1 Implementierung werden zunächst alle gewichteten Eingangswerte und deren Verzögerungen zusammengefasst und berechnet:

$$w[n] = b_0 \cdot x[n] + b_1 \cdot x[n-1] + b_2 \cdot x[n-2]$$

Anschließend wird die Rückkopplung der verzögerten Ausgangswerte realisiert:

$$y[n] = w[n] - a_1 \cdot y[n-1] - a_2 \cdot y[n-2]$$

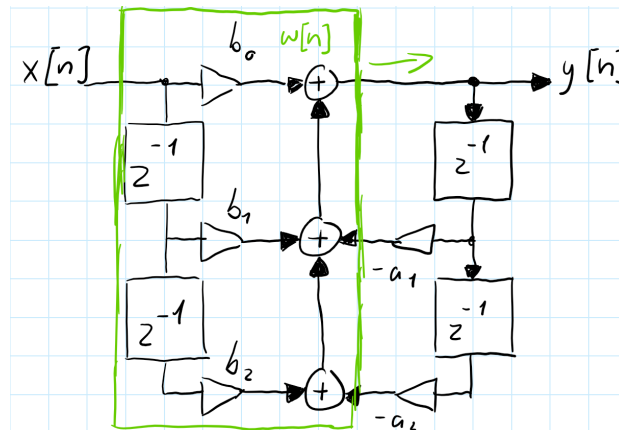


Abbildung 1: Signalflussbild: Direktform 1

In dieser Struktur werden vier Verzögerungselemente verwendet: zwei für die Eingangsverzögerung, $x[n-1]$ und $x[n-2]$, sowie zwei für die Ausgangsverzögerung, $y[n-1]$ und $y[n-2]$. Die Multiplikationen zwischen den Verzögerungselementen und den Koeffizienten erfolgen parallel, wodurch sich eine hohe Verarbeitungsgeschwindigkeit erzielen lässt.

Die Direktform 1 zeichnet sich durch ihre konzeptionelle Einfachheit und die direkte Korrespondenz zur Differenzengleichung aus. Nachteilhaft bei dieser Struktur ist der erhöhte Speicherbedarf, aufgrund der redundanten Verzögerungselemente.

Direktform 2 (Kanonische Form)

Die Direktform 2, oder auch als kanonische Form bekannt, ist eine optimierte Variante der Direktform 1, welche durch die Umordnung der Operationen die Anzahl der benötigten Verzögerungselemente minimiert.

Diese Struktur macht sich die Kommutativität (Vertauschbarkeit) linearer zeitinvarianter (LTI)-Systeme zunutze, indem sie die Reihenfolge von der Vorwärts- und Rückwärtsverarbeitung vertauscht.

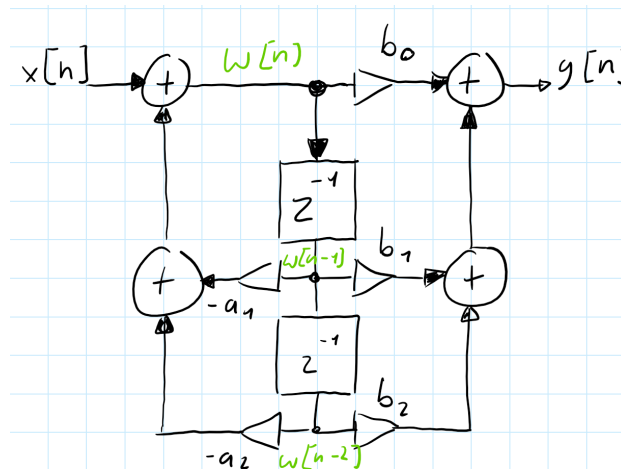


Abbildung 2: Signalflussbild: Direktform 2

Zunächst wird ein Zwischenwert $w[n]$ durch die rekursive Beziehungen erzeugt:

$$w[n] = x[n] - a_1 \cdot w[n-1] - a_2 \cdot w[n-2]$$

Der Zwischenwert $w[n]$ dient als Substituion für die Verzögerungselemente der Ein- und Ausgangswerte.

$$y[n] = b_0 \cdot w[n] + b_1 \cdot w[n-1] + b_2 \cdot w[n-2]$$

In dieser Struktur werden nur zwei Verzögerungselemente für $w[n-1]$ und $w[n-2]$ benötigt, da die rekursive als auch die nicht-rekursive Verarbeitung auf dieselben verzögerten Werte zugreifen.

Die Bezeichnung “kanonisch” bezieht sich darauf, dass diese Implementierung die minimal mögliche Anzahl von Verzögerungselementen verwendet und dadurch die Speichereffizienz optimiert.

Die Direktform 2 eignet sich Ideal für Systeme mit begrenzten Speicherressourcen. Jedoch weist diese Struktur eine erhöhte Empfindlichkeit gegenüber Quantisierungseffekten auf, da die Zwischenwerte $w[n]$ größere Dynamikbereiche aufweisen können als die ursprünglichen Ein- und Ausgangswerte.

Transponierte Direktform 2:

Die transponierte Direktform 2 kann gebildet werden mittels der Anwendung des Transpositionstheorems auf die Direktform 2. Dieses Theorem besagt, dass die Umkehrung aller Signalflussrichtungen bei gleichzeitiger Vertauschung von Eingängen und Ausgängen eine äquivalente Übertragungsfunktion ergibt. Die resultierende Struktur weist oft günstigere numerische Eigenschaften auf, als die ursprüngliche Form.

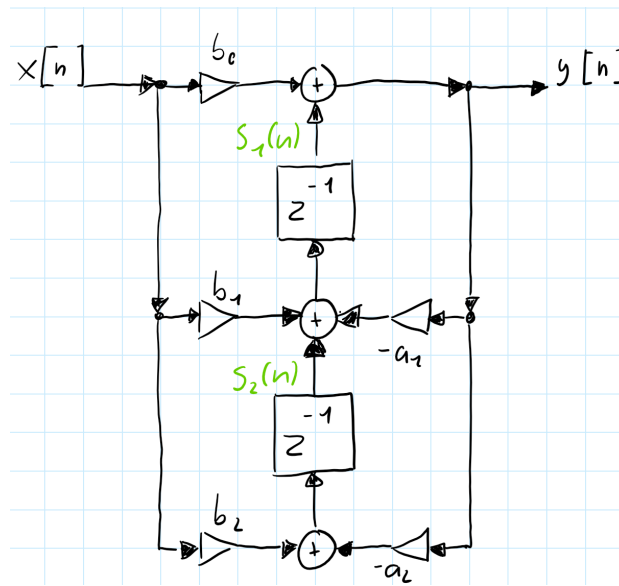


Abbildung 3: Signalflussbild: Transponierte Direktform 2

Die transponierte Direktform 2 wird durch die folgende Differenzgleichungen beschrieben:

$$s_2[n] = b_2 \cdot x[n] - a_2 \cdot y[n]$$

$$s_1[n] = s_2[n] + b_1 \cdot x[n] - a_1 \cdot y[n]$$

$$y[n] = s_1[n] + b_0 \cdot x[n]$$

Bei der transponierten Direktform 2 werden die Zwischenwerte $s_1[n]$ und $s_2[n]$ als Verzögerungselemente verwendet. Diese Struktur eignet sich besonders gut in der Implementierung von Echtzeitanwendung, aufgrund ihrer effizienten Berechnung und der guten Eignung für parallele oder pipelined Hardware-Architekturen.

Ein wesentlicher Vorteil der transponierten Direktform 2 liegt in ihrer reduzierten Empfindlichkeit gegenüber Koeffizientenquantisierung. Die Rückkopplungsschleifen sind kürzer als in der Direktform 2, was zu einer verbesserten numerischen Stabilität führt.

Implementierungserklärung der Biquad Filter in Arduino

Für die Implementierung der biquadratischen Filter in Arduino werden die Differenzengleichungen aus den Theoretischen Grundlagen entnommen und in Code umgesetzt.

Platzhalter

Filter Basisklasse

```
class Filter
{
public:
    virtual ~Filter() {}
    virtual float filter(float) = 0;
};
```

Direktform 1

```
class BiquadFilterDF1 : public Filter
{
private:
    const float b_0;
    const float b_1;
    const float b_2;
    const float a_1;
    const float a_2;

    float x_0 = 0;
    float x_1 = 0;
    float y_1 = 0;
    float y_2 = 0;
```

```

public:
    BiquadFilterDF1(const float (&b)[3], const float (&a)[3], float gain)
        : b_0(gain * b[0] / a[0]),
          b_1(gain * b[1] / a[0]),
          b_2(gain * b[2] / a[0]),
          a_1(a[1] / a[0]),
          a_2(a[2] / a[0])
    {
    }

    float filter(float x_0)
    {
        float x_2 = x_1;
        x_1 = x_0;

        float y_0 = b_0 * x_0 + b_1 * x_1 + b_2 * x_2 - a_1 * y_1 - a_2 * y_2;

        y_2 = y_1;
        y_1 = y_0;

        return y_0;
    }
};

```

Direktform 2

```

class BiquadFilterDF2 : public Filter
{
private:
    // Filterkoeffizienten
    const float b_0;
    const float b_1;
    const float b_2;
    const float a_1;
    const float a_2;

    float w_0 = 0;
    float w_1 = 0;

public:
    // Konstruktor mit Koeffizienten und Verstärkung (Gain)
    BiquadFilterDF2(const float (&b)[3], const float (&a)[3], float gain)

```

```

        : b_0(gain * b[0] / a[0]),
          b_1(gain * b[1] / a[0]),
          b_2(gain * b[2] / a[0]),
          a_1(a[1] / a[0]),
          a_2(a[2] / a[0])
    {
    }

float filter(float x_0)
{
    float w_2 = w_1;
    w_1 = w_0;
    w_0 = x_0 - a_1 * w_1 - a_2 * w_2;

    //Differenzengleichung der Direct form 2
    float y_0 = b_0 * w_0 + b_1 * w_1 + b_2 * w_2;

    return y_0;
}
};

```

```

class BiquadFilterTDF2 : public Filter
{
private:
    const float b_0;
    const float b_1;
    const float b_2;
    const float a_1;
    const float a_2;

    float s_1 = 0;
    float s_2 = 0;

public:
    BiquadFilterTDF2(const float (&b)[3], const float (&a)[3], float gain)
        : b_0(gain * b[0] / a[0]),
          b_1(gain * b[1] / a[0]),
          b_2(gain * b[2] / a[0]),
          a_1(a[1] / a[0]),
          a_2(a[2] / a[0])
    {
    }

    float filter(float x_0)

```

```

{
    float y_0 = s_1 + b_0 * x_0;

    s_1 = s_2 + b_1 * x_0 - a_1 * y_0;
    s_2 = b_2 * x_0 - a_2 * y_0;

    return y_0;
}
};

```

Koeffizienten

```

const float b_0 = f;
const float b_1 = f;
const float b_2 = f;
const float a_0 = f;
const float a_1 = f;
const float a_2 = f;

const float b_coefficients[] = { b_0, b_1, b_2};
const float a_coefficients[] = { a_0, a_1, a_2};
const float gain = 1;

```

```

BiquadFilterTDF2 biquad(b_coefficients, a_coefficients, gain);

```

Filter Anwendung

```

float filtered = biquad.filter(Input);

```